edexcel

Pearson Edexcel Level 1/Level 2 GCSE in Computer Science (1CP0)

Specification

First teaching September 2013

Pearson Education Limited is one of the UK's largest awarding organisations, offering academic and vocational qualifications and testing to schools, colleges, employers and other places of learning, both in the UK and internationally. Qualifications offered include GCSE, AS and A Level, NVQ and our BTEC suite of vocational qualifications, ranging from Entry Level to BTEC Higher National Diplomas. Pearson Education Limited administers BTEC qualifications.

Through initiatives such as onscreen marking and administration, Pearson is leading the way in using technology to modernise educational assessment, and to support teachers and students.

References to third-party material made in this specification are made in good faith. We do not endorse, approve or accept responsibility for the content of materials, which may be subject to change, or any opinions expressed therein. (Material may include textbooks, journals, magazines and other publications and websites.)

Publications Code UG031443

Introduction

The Pearson Edexcel Level 1/Level 2 GCSE in Computer Science is designed for use in schools and colleges. It is part of a suite of GCSE qualifications offered by Pearson.

About this specification

Rationale

The Pearson Edexcel Level 1/Level 2 GCSE in Computer Science has been developed in response to a number of recent initiatives aimed at promoting computer science as a rigorous, knowledge-based subject discipline that should be part of every young person's education.

These initiatives include:

- Recommendation 7 of the Royal Society report 'Shut down or restart? The way forward for computing in UK schools' (January 2012)
- 'Computer Science: A curriculum for schools' produced by the Computing at School (CAS) Working Group (March 2012)
- 'Computing: Programmes of study for Key Stages 1–4' (draft) published by the Department for Education (February 2013)

Content

The content of the Pearson Edexcel Level 1/Level 2 GCSE in Computer Science is based on and mapped against the Computer Science curriculum for schools produced by the CAS Working Group. See *Appendix 5* for further details.

The qualification is also benchmarked against the curricula of high-performing jurisdictions in other parts of the world, notably India, Israel and Singapore. See 'International Comparisons' a briefing note produced by CAS (November 2011).

Pearson has consulted widely with members of the computing community – employers, higher education institutions and teachers – to ensure that the content of this GCSE is relevant, fit for purpose and supports progression into higher education and ultimately into employment.

Assessment

While we recognise that programming is an important element of computer science, we feel strongly that the underlying principles of logic, decomposition, algorithms, data representation, communication etc. are even more fundamental and durable. This is reflected in the assessment model used for this qualification.

The written paper, 'Principles of Computer Science', is a rigorous, intellectually challenging examination with a weighting of 75% that requires a high level of computational thinking.

Practical programming skills are assessed in the controlled assessment, which has a weighting of 25%.

This is similar to the approach taken by high-performing jurisdictions in other parts of the world, such as India, Israel and Singapore.

Qualification objectives

The Pearson Edexcel Level 1/Level 2 GCSE in Computer Science aims to:

- develop an understanding of the fundamental principles of computer science
- encourage a systematic approach to solving problems using computational thinking skills
- provide practical experience of writing computer programs to solve problems and create purposeful artefacts
- foster an appreciation of current and evolving computer science and an understanding of the moral and ethical issues associated with them
- broaden educational and career opportunities in the computing, engineering or technology industries.

Contents

Specification at a glance

The Pearson Edexcel Level 1/Level 2 GCSE in Computer Science is a linear qualification. It has two assessment components:

- paper-based assessment: Principles of Computer Science
- controlled assessment: Practical Programming.

Paper-based assessment: Principles of Computer Science — ***Unit code: 1CP0/01**

Externally assessed
Availability: June
First assessment: June 2015

75% of the total GCSE

Overview of content

- Understanding and application of abstraction techniques: modelling, decomposing and generalising.
- Understanding and application of what an algorithm is and how an algorithm can be used to solve a problem.
- Understanding and application of how data is represented by a computer.
- Understanding the main components of a computer system and the computer architecture.
- How data is transported on the internet.

Overview of assessment

The assessment is paper based.

The assessment is 120 minutes.

The assessment consists of 5 questions.

The assessment consists of 90 marks.

Each question is set in a context, draws on topics from across the specification and is broken down into a number of parts.

A range of different question types is used, including extended writing that assesses Quality of Written Communication.

Controlled assessment: Practical Programming — ***Unit codes: 1CP0/2A, 1CP0/2B, 1CP0/2C**

Internally assessed
Availability: June
First assessment: June 2015

25% of the total GCSE

Overview of content

This is a practical 'making task' that enables students to demonstrate their computational techniques using a programming language. Students will:

- decompose problems into sub-problems
- create original algorithms or work with algorithms produced by others
- design, write, test, explain and correct errors in a program.

Controlled assessment: Practical Programming	***Unit codes: 1CP0/2A, 1CP0/2B, 1CP0/2C**

Overview of assessment

Controlled assessment activities will be released each January.

The assessment will be carried out at a computer under controlled conditions.

The controlled assessment may take place over multiple sessions up to a combined duration of 15 hours.

The assessment consists of 4 activities.

The assessment consists of 50 marks.

Students must select one programming language from the following:

- Python (unit code 1CP0/2A)
- Java (unit code 1CP0/2B)
- C-derived language (unit code 1CP0/2C).

Assessment includes extended writing that assesses Quality of Written Communication.

* See *Appendix 3* for a description of these codes and all other codes relevant to this qualification.

Qualification content

Topic 1: Problem solving

Students are expected to develop a set of computational thinking skills that enable them to understand how computer systems work and design, implement and analyse algorithms for solving problems.

Students should be given repeated opportunities to tackle computational problems of various sorts, including some substantial problem-solving tasks.

Subject content	Students should:
1.1 Decomposition	1.1.1 Be able to analyse a problem, investigate requirements [inputs, outputs, processing], specify success criteria and design solutions
	1.1.2 Be able to decompose a problem into smaller sub-problems
1.2. Algorithms	1.2.1 Understand what an algorithm is and what algorithms are used for
	1.2.2 Be able to create an algorithm to solve a particular problem making use of programming constructs [sequence, selection, repetition]
	1.2.3 Be able to describe the purpose of a given algorithm and explain how a simple algorithm works
	1.2.4 Be able to identify the correct output of an algorithm for a given set of data
	1.2.5 Be able to identify and correct errors in algorithms
	1.2.6 Be able to code an algorithm into a high-level language (from pseudocode, flow charts, structured English or written descriptions)*
	1.2.7 Understand how standard algorithms [quick sort, bubble sort, selection sort, linear search, binary search, breadth first search, depth first search, maximum/minimum, mean, count] work
	1.2.8 Understand factors that affect the efficiency of an algorithm [worst case, average case, best case]

* See *Appendix 2* for a key definitions related to this content.

Topic 2: Programming

Learning to program is a core component of a computer science course. Students should be competent at reading and writing programs and be able to reason about code. They must be able to apply their skills to solve real problems and produce robust programs.

Students should be given repeated opportunities to develop and practise their programming skills.

Subject content	Students should:
2.1 Develop code	2.1.1 Be able to write programs in a high-level programming language
	2.1.2 Understand the benefit of producing programs that are easy to read and be able to use techniques [comments, descriptive variable names, indentation] to improve readability and to explain how the code works
	2.1.3 Be able to differentiate between types of error in programs [logic, syntax, runtime]
	2.1.4 Be able to design and use test plans and test data
	2.1.5 Be able to interpret error messages, and identify, locate and fix errors in a program
	2.1.6 Be able to identify what value a variable will hold at a given point in a program [trace table]
	2.1.7 Be able to evaluate the strengths and weaknesses of a program and suggest improvements
	2.1.8 Be able to make effective use of tools offered in an integrated development environment [watcher, break points, single-step, step throughs]
2.2 Constructs	2.2.1 Be able to identify the structural components of a program [variable and type declarations, command sequences, conditionals, repetition, data structures, subprograms]
	2.2.2 Be able to use sequencing, selection and repetition constructs in their programs
2.3 Data types and structures	2.3.1 Understand the need for and be able to select and use data types [integer, real, Boolean, char]
	2.3.2 Understand the need for and be able to select and use data structures [arrays]
	2.3.3 Understand the need for and be able to manipulate strings
	2.3.4 Understand what variables and constants are and be able to give examples of the role of variables and constants in programs
	2.3.5 Be able to declare, initiate and assign values to and to correctly use global and local variables

Subject content	Students should:
2.4 Input/ output	2.4.1 Be able to write code that accepts and respond appropriately to user input
	2.4.2 Understand the need for validation and be able to program simple validation
	2.4.3 Be able to write code that outputs information to a screen and understand and use Cartesian x/y coordinates
	2.4.4 Be able to design and code a user interface
	2.4.5 Be able to write code that opens/closes, reads/writes, deletes, inserts, appends from/to a file
2.5 Operators	2.5.1 Understand the purpose of and be able to use arithmetic operators [plus, minus, divide, multiply, modulus, integer division]
	2.5.2 Understand the purpose of and be able to use relational operators [equal to, less than, greater than, not equal to, less than or equal to, greater than or equal to]
	2.5.3 Understand the purpose of and be able to use Boolean operators [AND, OR, NOT]
2.6 Subprograms	2.6.1 Understand the benefits of using subprograms and be able to write code that uses user-written and pre-existing [built-in, library] subprograms
	2.6.2 Understand the concept of passing data into and out of subprograms

Topic 3: Data

Computers are able to store and manipulate large quantities of data. They use binary to represent different types of data.

Students are expected to learn how different types of data are represented in a computer and should be given the opportunity to gain practical experience of using SQL to handle data stored in a structured database.

Subject content	Students should:
3.1 Binary	3.1.1 Understand that computers use binary to represent data and instructions
	3.1.2 Understand the terms bit, nibble, byte, kilobyte (KB), megabyte (MB), gigabyte (GB), terabyte (TB)
	3.1.3 Be able to convert between binary and denary whole numbers (0-255) and vice versa
	3.1.4 Be able to perform simple binary arithmetic [add, multiply] and understand the concept of overflow
	3.1.5 Understand why hexadecimal notation is used and be able to convert between hexadecimal and binary and vice versa
	3.1.6 Understand that file storage is measured in bytes and data transmission is measured in bits per seconds. Be able to calculate the time required to a transmit file
3.2 Data representation	3.2.1 Understand how computers represent and manipulate numbers [unsigned integers, real numbers]
	3.2.2 Understand how computers represent characters [ASCII, Unicode]
	3.2.3 Understand how bitmap images are represented in binary [pixels, resolution, colour depth]
	3.2.4 Understand how sound is represented in binary [sampling frequency, word length]
	3.2.5 Recognise the limitations of binary representation of data and how bit length constrains the range of values that can be represented
3.3 Compression	3.3.1 Understand the need for data compression and methods of compressing data [lossless, lossy] and that JPEG and MP3 are examples of lossy algorithms
	3.3.2 Understand how a lossless, run-length encoding [RLE] algorithm works
3.4 Encryption	3.4.1 Understand the need for data encryption
	3.4.2 Be able to demonstrate how a Caesar cipher algorithm works
3.5 Databases	3.5.1 Understand the characteristics of structured and unstructured databases
	3.5.2 Understand how data can be organised in a structured database [tables, records, fields, relationships, keys]
	3.5.3 Be able to use SQL statements *

* See *Appendix 2* for a key definitions related to this content.

Topic 4: Computers

Students must be familiar with the hardware and software components that make up a computer system and recognise that computers come in all shapes and sizes from embedded microprocessors to distributed clouds.

Students should be given the opportunity to gain practical experience of interpreting instructions written in assembly language.

Subject content	Students should:
4.1 Hardware	4.1.1 Understand the function of hardware components of a computer system [processor (CPU), memory, secondary storage, input, output] and explain how they work together
	4.1.2 Understand the concept of a stored program and the role of components of the processor [control unit (CU) arithmetic/logic unit (ALU), registers, clock, address bus, data bus] in the fetch-decode-execute cycle
	4.1.3 Be able to interpret a block of assembly code using a given set of commands *
	4.1.4 Understand that there is a range of different input and output devices available and be able to select appropriate input/output devices for a specified purpose
	4.1.5 Understand how data is stored on physical devices [magnetic, optical, solid state]
4.2 Logic	4.2.1 Be able to construct truth tables for a given logic statement [AND, OR, NOT]
	4.2.2 Be able to produce logic statements for a given problem
4.3 Software	4.3.1 Understand the functions of an operating system [file management, input/output, resource allocation, process management, network management, user management]
	4.3.2 Understand that application software such as a web browser, word processor, spreadsheet or apps are computer programs
4.4 Programming languages	4.4.1 Be able to compare features of high-level and low-level programming languages and assess their suitability for a particular task
	4.4.2 Understand what is meant by a compiler and an interpreter
	4.4.3 Understand how programming languages can be used to program a graphical interface (GUI), mobile phone apps and control devices [microcontrollers, sensors, actuators]

* See *Appendix 2* for a key definitions related to this content.

Topic 5: Communication and the internet

Computer networks and the internet are now ubiquitous. Many computer applications in use today would not be possible without networks. Students should understand the key principles behind the organisation and of computer networks. Ideally, they should be able to experiment by setting up a simple network

Students should be given the opportunity to gain practical experience of creating web pages.

Subject content	Students should:
5.1 Networks	5.1.1 Understand the advantages and disadvantages of connecting computers in a network
	5.1.2 Understand the different types of networks [LAN, WAN, PAN, VPN]
	5.1.3 Understand the network media [copper cable, fibre optic cable, wireless]
	5.1.4 Understand that network data speeds are measured in bits per second [Mbps, Gbps]
	5.1.5 Understand the role of and need for standard network protocols
	5.1.6 Understand that data is transmitted over networks using packets [TCP/IP]
	5.1.7 Understand that data transmitted over a network is subject to errors [interference, power spikes, interception, cross talk]
	5.1.8 Be familiar with the use of check sums to detect errors in data transmission
	5.1.9 Understand the concept of and need for network addressing and host names [MAC addresses]
	5.1.10 Understand network topologies [bus, ring, star, mesh]
5.2 The internet and the world wide web (WWW)	5.2.1 Understand the structure of the internet [clients, servers, routers, connecting backbone, domain names, URLs, ISP, HTTP]
	5.2.2 Understand the role of a web server
	5.2.3 Be able to use HTML and CSS to construct web pages [formatting, links, images, media, layout, styles, lists]
5.3 Client-server	5.3.1 Understand the client-server model
	5.3.2 Be able to differentiate between client-side and server-side processing

Topic 6: The bigger picture

Students should recognise that computer applications impact on nearly every aspect of the world in which they live. They should develop an informed opinion about issues relating to the application of computer science.

Students should understand what constitutes safe and responsible practice and adhere to it at all times.

Subject content	Students should:
6.1 Impact	6.1.1 Be able to discuss the impact of computer science on the economy, society and the environment
	6.1.2 Be able to assess the benefits and the consequences of implementing a new computer system/program
	6.1.3 Be aware of emerging trends in computer science
6.2 Legal and ethical	6.2.1 Be aware of the legal and ethical implications of computing in a networked world [privacy, security, access, data protection]
	6.2.2 Have an understanding of issues relating to ownership of software and digital artefacts [intellectual property, patents, licensing, open source and proprietary software]

Assessment

Assessment summary

Summary of table of assessment

Paper-based assessment: Principles of Computer Science	***Unit code: 1CP0/01**
Externally assessed Availability: June First assessment: June 2015	**75% of the total GCSE**

Overview of content

- Understanding and application of abstraction techniques: modelling, decomposing and generalising.
- Understanding and application of what an algorithm is and how an algorithm can be used to solve a problem.
- Understanding and application of how data is represented by a computer.
- Understanding the main components of a computer system and the computer architecture.
- How data is transported on the internet.

Overview of assessment

The assessment is paper based.

The assessment is 120 minutes.

The assessment consists of 5 questions.

The assessment consists of 90 marks.

Each question is set in a context, draws on topics from across the specification and is broken down into a number of parts.

A range of different question types is used, including extended writing that assesses Quality of Written Communication.

Controlled assessment: Practical Programming	***Unit codes: 1CP0/2A, 1CP0/2B, 1CP0/2C**
Internally assessed Availability: June First assessment: June 2015	**25% of the total GCSE**

Overview of content

This is a practical 'making task' that enables students to demonstrate their computational techniques using a programming language. Students will:

- decompose problems into sub-problems
- create original algorithms or work with algorithms produced by others
- design, write, test, explain and correct errors in a program.

Controlled assessment: Practical Programming	***Unit codes: 1CP0/2A, 1CP0/2B, 1CP0/2C**

Overview of assessment

Controlled assessment activities will be released each January.

The assessment will be carried out at a computer under controlled conditions.

The controlled assessment may take place over multiple sessions up to a combined duration of 15 hours.

The assessment consists of 4 activities.

The assessment consists of 50 marks.

Students must select one programming language from the following:

- Python (unit code 1CP0/2A)
- Java (unit code 1CP0/2B)
- C-derived language (unit code 1CP0/2C).

Assessment includes extended writing that assesses Quality of Written Communication.

*See *Appendix 3* for a description of these codes and all other codes relevant to this qualification.

Assessment Objectives and weightings

Students must:		% in GCSE
A01	Recall and demonstrate knowledge and understanding of computer science	32-42%
A02	Apply knowledge and understanding, solve problems and develop solutions	40-50%
A03	Evaluate, make reasoned judgements and draw conclusions	15-25%
	TOTAL	100%

Relationship of Assessment Objectives to papers

Unit	Assessment Objective			
	AO1	AO2	AO3	Total for AO1, AO2 and AO3
Paper 1: **Principles of Computer Science**	27%-32%	30%-35%	10%-15%	75%
Controlled assessment: **Practical Programming**	5%-10%	10%-15%	5%-10%	25%
Total for GCSE	32-42%	40-50%	15-25%	100%

Entering your students for assessment

Student entry

Details of how to enter students for the examinations for this qualification can be found in the Edexcel *Information Manual*, a copy is sent to all examinations officers. The information can also be found on our website: www.edexcel.com

Students are required to sit all of their examinations at the end of the course. Students may complete the controlled assessment task(s) at any appropriate point during the course and controlled assessment work must be submitted for moderation at the end of the course. Centres must ensure that controlled assessment tasks submitted are valid for the series in which they are submitted.

Forbidden combinations and classification code

Centres should be aware that students who enter for more than one GCSE qualification with the same classification code will have only one grade (the highest) counted for the purpose of the School and College Performance Tables (please see Appendix 3).

Students should be advised that, if they take two qualifications with the same classification code, schools and colleges are very likely to take the view that they have achieved only one of the two GCSEs. The same view may be taken if students take two GCSE qualifications that have different classification codes but have significant overlap of content. Students who have any doubts about their subject combinations should check with the institution to which they wish to progress before embarking on their programmes.

Access arrangements and special requirements

Pearson's policy on access arrangements and special considerations for GCE, GCSE, and Entry Level is designed to ensure equal access to qualifications for all students (in compliance with the Equality Act 2010) without compromising the assessment of skills, knowledge, understanding or competence.

Please see the Joint Council for Qualifications (JCQ) website (www.jcq.org.uk) for its policy on access arrangements, reasonable adjustments and special considerations.

Please see our website (www.edexcel.com) for:

- the forms to submit for requests for access arrangements and special considerations
- dates to submit the forms.

Requests for access arrangements and special considerations must be addressed to:

Special Requirements
Pearson Education Limited
One90 High Holborn
London WC1V 7BH

Equality Act 2010

Please see our website (www.edexcel.com) for information on the Equality Act 2010.

Assessing your students

The first assessment opportunity for this qualification will take place in the June 2015 series and in each following June series for the lifetime of the specification.

Your student assessment opportunities

Unit	June 2015	June 2016
Paper 1	✓	✓
Controlled assessment	✓	✓

All assessments must be taken at the end of the course.

Awarding and reporting

The grading, awarding and certification of this qualification will comply with the requirements of the current GCSE/GCE Code of Practice, which is published by the Office of Qualifications and Examinations Regulation (Ofqual). The GCSE qualification will be graded and certificated on an eight-grade scale from A* to G.

The first certification opportunity for the Pearson Edexcel Level 1/Level 2 GCSE in Computer Science will be 2015.

Students whose level of achievement is below the minimum judged by Pearson to be of sufficient standard to be recorded on a certificate will receive an unclassified U result.

Stretch and challenge

Students can be stretched and challenged in all units through the use of different assessment strategies, for example:

- using a variety of stems in questions, for example analyse, evaluate, discuss, compare
- ensuring connectivity between sections of questions
- a requirement for extended writing
- using a wider range of question types to address different skills, for example open-ended questions, case studies.

Malpractice and plagiarism

For up-to-date advice on malpractice and plagiarism, please refer to the latest Joint Council for Qualifications (JCQ) *Instructions for Conducting Coursework* document. This document is available on the JCQ website: www.jcq.org.uk.

For additional information on malpractice, please refer to the latest Joint Council for Qualifications (JCQ) *Suspected Malpractice in Examinations and Assessments: Policies and Procedures* document, available on the JCQ website.

Student recruitment

Pearson's access policy concerning recruitment to our qualifications is that:

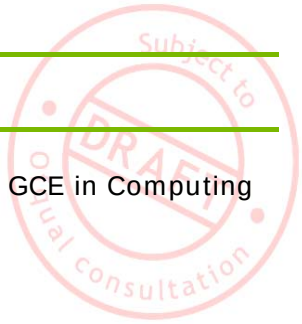
- they must be available to anyone who is capable of reaching the required standard
- they must be free from barriers that restrict access and progression
- equal opportunities exist for all students.

Prior knowledge and skills

There are no prior learning requirements for this qualification.

Progression

Students can progress from this qualification onto a GCE in Computing or to the Pearson BTEC Nationals in ICT.



Grade descriptions

A	Students recall, select and communicate precise knowledge and detailed understanding of computer science. They demonstrate a comprehensive understanding of how computers and computer systems work, They use technical knowledge, terminology and conventions appropriately and consistently. They apply appropriate computational thinking skills to systematically analyse, model and solve problems. They use a wide range of computational concepts and practices appropriately to develop effective solutions. They critically evaluate and draw detailed, informed conclusions about the way they and others use computer science to solve problems, making reasoned judgements about the effectiveness of solutions and suggesting realistic improvements.
C	Students recall, select and communicate a secure knowledge and understanding of computer science. They demonstrate understanding of how computers and computer systems work. They use technical knowledge, terminology and conventions appropriately. They apply some appropriate computational thinking skills to analyse, model and solve problems. They use a range of computational concepts and practices appropriately to develop solutions. They evaluate and draw conclusions about the way they and others use computer science to solve problems, drawing conclusions about the effectiveness of solutions and suggesting some realistic improvements.
F	Students recall, select and communicate a basic knowledge and understanding of aspects of computer science. They have a limited understanding of how computers and computer systems work. They use limited technical knowledge, terminology and conventions. They apply limited computational thinking skills to analyse, model and solve straightforward problems. They use a limited range of computational concepts and practices appropriately to develop basic solutions. They comment on the way they and others use computer science to solve problems, drawing elementary conclusions about the effectiveness of solutions and suggesting some simple improvements.

Controlled assessment

Controlled assessment summary

Overview

The purpose of the controlled assessment is to test student skills in responding to computer science problems. In the controlled assessment, evidence of skills will be demonstrated through responses to a number of activities set in a context.

The controlled assessment consists of four activities:

- Activity 1: Producing a working solution from a given design
- Activity 2: Designing and developing
- Activity 3: Programming and testing
- Activity 4: Evaluation
- Students may spend no more than 15 hours working on the controlled assessment activities
- The controlled assessment brief specifies the recommended duration which students should spend on each activity
- Students may base their answer on any one of the supported programming languages specified by Pearson (at the time of publication, these are Python, Java, and any C-derived language, please visit www.edexcel.com for more information)
- Levels of control:
  - high task setting
  - medium task taking
  - medium task marking.

Students must select one programming language from the following:

- Python (unit code 1CP0/2A)
- Java (unit code 1CP0/2B)
- C-derived language (unit code 1CP0/2C)

Levels of control

Task setting — High control

Students must respond to a task set by Pearson. The controlled assessment task will be made available on the Edexcel website for teachers to download in January of the terminal year.

The format of the activities will be similar each year but the context will change. Teachers must ensure that students are completing the correct controlled assessment task for their terminal year. The front sheet shows the dates for which it is valid.

Task taking — Medium controls

Authenticity control

Students must work alone to develop a response to the task. Students must sign the Controlled Assessment Authentication Statement (*Appendix 4*).

Collaboration control

Students may not work with others to develop a response to the task.

Feedback control

Teachers may help students to understand rubrics, assessment criteria and controlled conditions. Teachers may not provide solutions to students. Any additional feedback must be recorded on the Controlled Assessment Authentication Statement (*Appendix 4*).

Resources control

Every student must each have equal access to IT resources. Internet access is not allowed. Where students are taking the controlled assessment over a number of sessions, at the end of each session their work must be saved and kept securely. Students must not take their work out of the controlled assessment setting.

Time control

15 hours' total duration is permitted for students to complete the assessment. This time may be divided up into sessions.

Task marking — Medium controls

Teachers should mark the controlled assessment using the assessment criteria on the following pages. There is no requirement to annotate students' work, although it is necessary to write full justification comments on the Controlled Assessment Authentication Statement (*Appendix 4*).

Students may provide solutions to the tasks in one of the programming languages specified by Pearson. Assessment criteria are applicable to all these languages.

Quality of Written Communication

Quality of Written Communication (QWC) will be assessed in the task. It will assess students on their ability to:

- present relevant information in a form that suits its purpose
- ensure that text is legible and that spelling, punctuation and grammar are accurate, so that meaning is clear
- use suitable structure and style of writing
- use specialist vocabulary when appropriate.

Presentation of work

Students must present their work for the controlled assessment electronically. Student files should be identified by student name and task and presented in a single folder.

Marking, standardisation and moderation

The controlled assessment is marked by centre staff. Where marking for this specification has been carried out by more than one teacher in a centre, there must be a process of internal standardisation carried out to ensure that there is a consistent application of the criteria laid down in the marking grids, across all the units.

Marks awarded by the centre will be subject to external moderation by Edexcel. This is to ensure consistency with national standards. Following the submission of marks, Pearson will notify centres of the students whose responses have been selected for moderation.

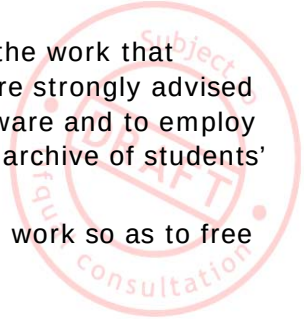
Work must be submitted in an approved digital format.

If the moderation indicates that centre assessment does not reflect national standards, an adjustment will be made to students' final marks to compensate for this.

Security and backups

It is the responsibility of the centre to keep secure the work that students have submitted for assessment. Centres are strongly advised to utilise firewall protection and virus checking software and to employ an effective backup strategy, so that an up-to-date archive of students' evidence is maintained.

Centres are advised to archive completed, assessed work so as to free up work space for work in progress.



Language of assessment

Assessment of these units will be available in English. All student work must be in English.

Further information

For more information on annotation, authentication, mark submission and moderation procedures, please refer to *Moderation of Controlled Assessments: Guidance for Centres for Pearson Edexcel Level 1/Level 2 GCSE in Computer Science*, available on the Edexcel website.

For up-to-date advice on teacher involvement, please refer to the Joint Council for Qualifications (JCQ*) Instructions for conducting coursework* document on the JCQ website: www.jcq.org.uk.

For up-to-date advice on malpractice and plagiarism, please refer to the Joint Council for Qualifications (JCQ) *Suspected Malpractice in Examinations and Assessments* document on the JCQ website (www.jcq.org.uk).

Controlled assessment criteria

Teachers must mark students' work using the assessment criteria specified below.

Activity number		Target: AO1 / AO2	Mark
1		**Producing a working solution from a given design**	6 marks
Level	**Mark**	**Descriptor**	
	0	No rewardable material.	
1	1–2	Program runs with some errors and only partially addresses the requirements of the brief. Some of the programming constructs selected are not appropriate to the design. Variable names, layout and structure do not aid readability. Comments do not explain how the program works.	
2	3–4	Program runs without errors and addresses most of the requirements of the brief. The programming constructs selected are appropriate to the design. Variable names, layout and structure make parts of the program easy to read. Comments partially explain how the program works.	
3	5–6	Program runs without errors and fully addresses all the requirements of the brief. The programming constructs selected are appropriate to the design and together produce an efficient solution. Variable names, layout and structure make the whole program easy to read. Comments fully explain how the program works.	

Activity number		Target AO1/AO2	Mark
2		**Designing and developing: The design document**	5 marks
Level	**Mark**	**Descriptor**	
	0	No rewardable material.	
1	1	Success criteria take little account of specified requirements. Limited attempt to decompose the problem into subprograms.	
2	2–3	Success criteria do not fully take account of specified requirements. The problem is partially decomposed into subprograms with some attempt to show the flow of data.	
3	4–5	Success criteria take account of all specified requirements. The problem is fully decomposed into subprograms and the flow of data is clearly shown.	

Activity number		Target: AO1/AO2	Mark
2		**Designing and developing: Programming the solution**	12 marks
Level	**Mark**	**Descriptor**	
	0	No rewardable material.	
1	1–4	Program runs with some errors and only partially addresses the requirements of the brief. Some of the programming constructs selected are not appropriate to the design. Variable names, layout and structure do not aid readability. Comments do not explain how the program works.	
2	5–8	Program runs without errors and addresses most of the requirements of the brief. The programming constructs selected are appropriate to the design. Variable names, layout and structure make parts of the program easy to read. Comments partially explain how the program works.	
3	9–12	Program runs without errors and fully addresses all the requirements of the brief. The programming constructs selected are appropriate to the design and together produce an efficient solution. Variable names, layout and structure make the whole program easy to read. Comments fully explain how the program works.	

Activity number		Target: AO1 / AO2	Mark
3		**Developing and testing: Programming the solution**	12 marks
Level	**Mark**	**Descriptor**	
	0	No rewardable material.	
1	1–4	The program is poorly designed. It runs with some errors and only partially addresses the requirements of the brief. Some of the programming constructs selected are not appropriate to the design. Variable names, layout and structure do not aid readability. Comments do not explain how the program works.	
2	5–8	Some aspects of the program are well designed. It runs without errors and addresses most of the requirements of the brief. The programming constructs selected are appropriate to the design. Variable names, layout and structure make parts of the program easy to read. Comments partially explain how the program works.	
3	9–12	The whole program is well designed. It runs without errors and fully addresses the requirements of the brief. The programming constructs selected are appropriate to the design and together produce an efficient solution. Variable names, layout and structure make the whole program easy to read. Comments fully explain how the program works.	

Activity number		Target: AO2	Mark
3		**Developing and testing: Test plan**	5 marks
Level	**Mark**	**Descriptor**	
	0	No rewardable material.	
1	1	The test plan contains little detail and does not test the main aspects of the program. Little or no use of test data.	
2	2–3	The test plan covers some aspects of the program. Test data is incomplete or poorly designed.	
3	4–5	A detailed test plan covers all aspects of the program. Effective test data has been designed.	

Activity number		Target: AO3	Mark
4		**Evaluation**	10 marks
Level	**Mark**	**Descriptor**	
	0	No rewardable material.	
1	1–4	Some strengths and weaknesses of each program are identified. Everyday language has been used and response lacks clarity and organisation. Spelling, punctuation and the rules of grammar are used with limited accuracy.	
2	5–7	A description of the strengths and weaknesses of each program illustrated with extracts of code. Some specialist technical terms used and the response shows focus and organisation. Spelling and punctuation and the rules of grammar used with some accuracy.	
3	8–10	An evaluation of the strengths and weaknesses of each program, illustrated with well-chosen extracts of code. Specialist technical terms used appropriately and the response shows good focus and organisation. Spelling, punctuation and the rules of grammar are used with considerable accuracy.	

Resources, support and training

Pearson resources

Pearson aims to provide and endorse the most comprehensive support for our qualifications.

Pearson publications

You can order further copies of the Specification and Sample Assessment Materials (SAMs) documents from:
Edexcel Publications
Adamsway
Mansfield
Nottinghamshire
NG18 4FN

Telephone: 01623 467 467
Fax: 01623 450 481
Email: publication.orders@edexcel.com
Website: www.edexcel.com

Endorsed resources

Pearson also endorses some additional materials written to support this qualification. Any resources bearing the Pearson Edexcel logo have been through a quality assurance process to ensure complete and accurate support for the specification. For up-to-date information about endorsed resources, please visit www.edexcel.com/endorsed

Please note that while resources are checked at the time of publication, materials may be withdrawn from circulation and website locations may change.

Pearson support services

Pearson has a wide range of support services to help you implement this qualification successfully.

ResultsPlus – ResultsPlus is an application launched by Pearson to help subject teachers, senior management teams, and students by providing detailed analysis of examination performance. Reports that compare performance between subjects, classes, your centre and similar centres can be generated in 'one-click'. Skills maps that show performance according to the specification topic being tested are available for some subjects. For further information about which subjects will be analysed through ResultsPlus, and for information on how to access and use the service, please visit www.edexcel.com/resultsplus

Ask the Expert – to make it easier for our teachers to ask us subject specific questions we have provided the **Ask the Expert** Service. This easy-to-use web query form will allow you to ask any question about the delivery or teaching of Pearson qualifications. You'll get a personal response, from one of our administrative or teaching experts, sent to the email address you provide. You can access this service at www.edexcel.com/ask

Support for students

Learning flourishes when students take an active interest in their education; when they have all the information they need to make the right decisions about their futures. With the help of feedback from students and their teachers, we've developed a website for students that will help them:

- understand subject specifications
- access past papers and mark schemes
- learn about other students' experiences at university, on their travels and when entering the workplace.

We're committed to regularly updating and improving our online services for students. The most valuable service we can provide is helping schools and colleges unlock the potential of their students. www.edexcel.com/students

Training

A programme of professional development and training courses, covering various aspects of the specification and examination, will be arranged by Pearson each year on a regional basis. Full details can be obtained from:

Training from Edexcel
Pearson Education Limited
One90 High Holborn
London
WC1V 7BH

Telephone: 0844 576 0027
Email: trainingbookings@pearson.com
Website: www.edexcel.com

Appendices

Appendix 1 Wider curriculum

Signposting

Issue	Topic 1	Topic 2	Topic 3	Topic 4	Topic 5	Topic 6
Spiritual						
Moral			✓			✓
Ethical			✓			✓
Social			✓			✓
Citizenship			✓			✓
Environmental						✓

Development suggestions

Issue	Topic	Opportunities for development or internal assessment
Moral	3 6	• acknowledging sources and respecting copyright when developing digital products • exploring the legislation relating to the use of ICT, including copyright and data protection • knowing the importance of incorporating ethical and legal principles into design techniques • exploring ways of minimising/mitigating the environmental impact of digital devices
Ethical	3 6	• acknowledging sources and respecting copyright when developing digital products • exploring issues that arise when information is transmitted and stored digitally • knowing the importance of incorporating ethical and legal principles into design techniques
Social	3 6	• acknowledging sources and respecting copyright when developing digital products • exploring issues that arise when information is transmitted and stored digitally • knowing the importance of incorporating ethical and legal principles into design techniques
Citizenship	3 6	• acknowledging sources and respecting copyright when developing digital products • knowing the importance of incorporating ethical and legal principles into design techniques
Environmental	6	• exploring ways of minimising/mitigating the environmental impact of digital devices

Appendix 2 Key definitions

Pseudocode Command Set

Questions in the written examination that involve code will use this pseudocode for clarity and consistency. However, students may answer questions using any valid method.

Data types

INTEGER

REAL

BOOLEAN

CHARACTER

Data structures

ARRAY

STRING

Identifiers

Identifiers are sequences of letters, digits and "_", starting with a letter, e.g. MyValue, My_Value, Counter2

Functions

LENGTH

RANDOM

Variables and arrays		
Syntax	**Description**	**Example**
SET Variable TO <value>	Assigns a value to a variable.	SET Counter TO 0 SET MyString TO ”Hello world”
SET Variable TO <expression>	Computes the value of an expression and assigns to a variable.	SET Sum TO Score + 10 SET Size to LENGTH(Word)
SET Array[index] TO <value>	Assigns a value to an element of a one-dimensional array, where index is an integer value starting at 0.	SET ArrayClass[1] TO “Ann” SET ArrayMarks[3]TO 56
SET Array TO [<value>, …]	Initialises a one-dimensional array with a set of values .	SET ArrayValues TO [1, 2, 3, 4, 5]
SET Array [RowIndex, ColumnIndex] TO <value>	Assigns a value to an element of a two dimensional array, where RowIndex and ColumnIndex are integer values starting at O.	SET ArrayClassMarks[2,4] TO 92

Selection		
Syntax	**Description**	**Example**

IF <expression> THEN
<command>
END IF
	If <expression> is true then command is executed.	IF Answer = 10 THEN
		SET Score TO Score + 1
		END IF

IF <expression> THEN
<command>
ELSE
<command>
END IF
	If <expression> is true then first <command> is executed, otherwise second <command> is executed.	IF Answer = 'correct' THEN
		SEND "Well done" TO DISPLAY
		ELSE
		SEND "Try again" TO DISPLAY
		END IF

Repetition		
Syntax	**Description**	**Example**

WHILE <condition> DO
<command>
END WHILE
	Pre-conditioned loop. Executes <command> whilst <condition> is true.	WHILE Flag = 0 DO
		SEND “All well” TO DISPLAY
		END WHILE

REPEAT <command> UNTIL <expression>	Post-conditioned loop. Executes <command> until <condition> is true. The loop must execute at least once.	REPEAT SET Go TO Go + 1 UNTIL Go = 10
REPEAT <expression> TIMES <command> END REPEAT	Count controlled loop. The number of times <command> is executed is determined by the expression	REPEAT 100-Number TIMES SEND “* ” TO DISPLAY END REPEAT

FOR <id> FROM <expression> TO <expression> DO
<command>
END FOR
	Count controlled loop. Executes <command> a fixed number of times.	FOR Index FROM 1 TO 10 DO
		SEND ArrayNumbers[Index] TO DISPLAY
		END FOR

FOR <id> FROM <expression> TO <expression> STEP <expression> DO
<command>
END FOR
	Count controlled loop using a step.	

FOR EACH <id> FROM <expression> DO <command> END FOREACH		SET WordsArray TO [“The”, “Sky”, “is”, “grey” SET Sentence to “” FOR EACH Word FROM WordUArray DO SET Sentence TO Sentence & Word & “ “ END FOREACH

Input/output		
Syntax	**Description**	**Example**
SEND <expression> TO DISPLAY	Sends output to the screen.	SEND “Have a good day.” TO DISPLAY
RECEIVE <identifier> FROM (type) <device>	Reads input of specified type.	RECEIVE Name FROM (STRING) KEYBOARD RECEIVE LengthOfJourney FROM (INTEGER) CARD_READER RECEIVE YesNo FROM (CHARACTER) CARD-_READER

File handling		
Syntax	**Description**	**Example**
READ <File> <record>	Reads in a record from a <file> and assigns to a <variable>. Each READ statement reads a record from the file.	READ MyFile.doc Record
WRITE <File> <record>	Writes a record to a file Each WRITE statement writes a record to the file.	WRITE MyFile.doc Answer1, Answer2, “xyz 01”

Subprograms		
Syntax	**Description**	**Example**
SUBPROGRAM <Name> (<parameter>, …) <command> END SUB	Defines a subprogram	SUBPROGRAM Add(Var1, Var2) Ans = Var 1 + Var2 RETURN Ans END SUB

Subprograms		
Syntax	**Description**	**Example**
RETURN <value>	Returns a value from a subprogram.	See above.
SUBPROG <id> (<parameter>, …)	Executes a subprogram.	SET SUBPROG IdentityKnown TO InList(“John Smith”)

Arithmetic operators	
Symbol	**Description**
+	Add
-	Subtract
/	Divide
*	Multiply
^	Exponent
MOD	Modulo
\	Integer division

Relational operators	
Symbol	**Description**
=	equal to
< >	not equal to
>	greater than
> =	greater than or equal to
<	less than
< =	less than or equal to

Logical operators	
Symbol	**Description**
AND	Returns true if both conditions are true.
OR	Returns true if any of the conditions are true.
NOT	Reverses the outcome of the expression; true becomes false, false becomes true.

Assembly language command set

Students are not expected to be able to write programs in assembly code. However, they should to be able to explain what a given block of assembly code does.

The table below lists the instructions students need to be familiar with. There is no need for them to memorise this table since an appropriate extract from it will be reproduced in the examination paper.

Instructions		
Operation	**Assembler**	**Action**
Add	ADD {condition} Rd, Rn, <value>	Adds <value> to the contents of Register n and stores the result in Register d.
	ADD {condition} Rd, Rn, Rm	Adds the contents of Register m to the contents of Register n and stores the result in Register d.
Subtract	SUB {condition} Rd, Rn, <value>	Subtracts <value> from the contents of Register n and stores the result in Register d.
	SUB {condition} Rd, Rn, Rm	Subtracts the contents of Register m from the contents of Register n and stores the result in Register d.
Mulitply	MUL {condition} Rd, Rm, Rs	Multiplies the contents of Register m by the contents of Register s and stores the result in Register d.
Move	MOV {condition} Rd, <value>	Moves <value> into Register d.
	MOV {condition} Rd, Rm	Moves the contents of Register m into Register d.
Compare	CMP {condition} Rn, <value>	Compares <value> with the value in Register n.
	CMP {condition} Rn, Rm	Compares the value in Register m with the value in Register n.
Load	LDR {condition} Rd, [Rm]	Loads the contents of the memory address stored in Register m into Register d.
	LDR {condition} Rd, <value>	Loads <value> into Register d.

Instructions		
Operation	**Assembler**	**Action**
Branch	B {condition} label	Branch
	BL {condition} label	Branch and store return address in Register d.
Store	STR {condition} Rd, <value>	Stores <value> in Register d.
	STR {condition} Rd, Rm	Stores contents of Register m in Register d.

Conditions	
Mnemonic	**Description**
EQ	Equal
NE	Not equal
GE	Greater than or equal
GT	Greater than
LE	Less than or equal
LT	Less than

SQL commands

Command	Description
SELECT (ColumnName, …) FROM TableName [WHERE <condition>] [ORDER BY (ColumnName, …)];	Retrieves data from a table. (The asterisk * can be used if all columns are required in the output.)
SELECT (ColumnName, …) FROM TableName, TableName, … WHERE <join criteria>] [ORDER BY (ColumnName, …)];	Retrieves data from two related table.
UPDATE TableName SET (ColumnName =Value, …) WHERE <condition>;	Updates rows in a database table.
DELETE FROM TableName [WHERE <condition>];	Deletes rows from a database table.
CREATE TABLE TableName (ColumnName data type, ….);	Creates a database table.
INSERT INTO TableName VALUES (Value, ….);	Inserts rows into a database table.
SELECT ColumnNam) FROM TableName WHERE ColumnName LIKE pattern;	Used to search for a specified pattern in a column.

Items in [ ] brackets are optional.

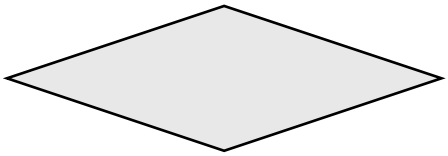
Ellipsis (ie '…') mean the preceding item may be repeated.

Comparison operators	
Symbol	**Description**
=	Equal
< >	Not equal
>	Greater than
> =	Greater than or equal
<	Less than
< =	Less than or equal

Flowchart symbols

Denotes the beginning and end of an algorithm.

Denotes a process to be carried out.



Denotes a decision to be made.

Shows the logical flow of the program.

Appendix 3 Codes

Type of code	Use of code	Code number
National classification codes	Every qualification is assigned to a national classification code indicating the subject area to which it belongs. Centres should be aware that students who enter for more than one GCSE qualification with the same classification code will have only one grade (the highest) counted for the purpose of the school and college performance tables.	2610
National Qualifications Framework (NQF) codes	Each qualification title is allocated a Ofqual National Qualifications Framework (NQF) code. The National Qualifications Framework (NQF) code is known as a Qualification Number (QN). This is the code that features in the DfE Section 96 and on the LARA as being eligible for 16-18 and 19+ funding, and is to be used for all qualification funding purposes. The QN is the number that will appear on the student's final certification documentation.	The QN for the qualification in this publication is: [Ofqual to provide.] GCSE – xxx/xxxx/x
Paper codes	Each paper is assigned a unit code. This paper code is used as an entry code to indicate that a student wishes to take the assessment for that unit. Centres will need to use the entry codes only when entering students for their examination.	Paper 1: - 1CP0/01 Controlled Assessment: - 1CP0/2A - 1CP0/2B - 1CP0/2C
Cash-in codes	The cash-in code is used as an entry code to aggregate the student's unit scores to obtain the overall grade for the qualification. Centres will need to use the entry codes only when claiming students' qualifications.	GCSE – 1CP0

Appendix 4 Authentication sheet

Controlled Assessment Authentication Statement

This sheet must be completed by the candidate and provided for work submitted for assessment.

Candidate name:		Assessor name:
Date issued:	Completion date:	Submitted on:
Qualification:		
Assessment reference and title:		

Please list the evidence submitted for each task. Indicate the page numbers where the evidence can be found or describe the nature of the evidence (e.g. video, illustration).

Task ref.	Evidence submitted	Page numbers or description
Comments for note by the Assessor:		

Candidate declaration

I certify that the work submitted for this assignment is my own. I have clearly referenced any sources used in the work. I understand that false declaration is a form of malpractice.

Candidate signature: Date:

_____

Appendix 5 Mapping between CAS and Pearson Edexcel GCSE in Computer Science

CAS curriculum	Location in Pearson Edexcel GCSE
2 - Key concepts	
2.1 Languages, machines and computation	Topic 1: sections 1.1, 1.2 Topic 2: sections 2.1, 2.2, 2.3, 2.4, 2.5, 2.6 Topic 4: sections 4.1, 4.2, 4.3
2.2 Data and representation	Topic 2: section 2.3 Topic 3: sections 3.1, 3.2, 3.3, 3.4, 3.5
2.3 Communication and co-ordination	Topic 5: sections 5.1, 5.2, 5.3
2.4 Abstraction and design	Topic 1: sections 1.1, 1.2 Topic 4: sections 4.1, 4.2, 4.3, 4.4
2.5 Computers and computing are part of a wider context	Topic 6: sections 6.1, 6.2
3 - Key processes: computational thinking	
3.1 Abstraction: modelling, decomposing and generalising	Topic 1: sections 1.1, 1.2
3.2 Programming	Topic 2: sections 2.1, 2.2, 2.3, 2.4, 2.5, 2.6 Topic 3: section 3.5 Topic 4: section 4.4 Topic 5: section 5.2
4 - Range and content: what a pupil should know	
4.1 Algorithms	Topic 1: section 1.2 Topic 2: section 2.1
4.2 Programs	Topic 1: sections 1.1, 1.2 Topic 2: sections 2.1, 2.2, 2.3, 2.4, 2.5, 2.6
4.3 Data	Topic 2: section 2.3 Topic 3: sections 3.1, 3.2, 3.3, 3.4, 3.5
4.4 Computers	Topic 4: sections 4.1, 4.2, 4.3, 4.4
4.5 Communication and the internet	Topic 5: sections 5.1, 5.2, 5.3

Subject to
DRAFT
Ofqual consultation

Further copies of this publication are available from
Edexcel Publications, Adamsway, Mansfield, Notts NG18 4FN

Telephone 01623 467467
Fax 01623 450481
Email: publication.orders@edexcel.com

Publications Code UG031443

For more information on Pearson Edexcel and Pearson BTEC qualifications please visit our website: www.pearson.com

Pearson Education Limited. Registered in England and Wales No. 872828
Registered Office: Edinburgh Gate, Harlow, Essex CM20 2JE. VAT Reg No GB 278 537121